

P1629R0

Standard Text Encoding

Published Proposal, 2019-06-17

Author:

[JeanHeyd Meneide](#)

Audience:

EWG, LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Latest:

https://thephd.github.io/vendor/future_cxx/papers/d1629.html

Abstract

The standard lacks facilities for transliterating and transcoding text from one form into another, leaving a serious barrier to entry for individuals who want to process text in any sensible manner in the Standard Library. This paper explores and proposes a static interface for encoding that can be used and built upon for the creation of higher-level abstractions.

Table of Contents

- 1 Revision History**
 - 1.1 Revision 0 - June 17th, 2019
- 2 Motivation**
- 3 Design**
 - 3.1 High Level
 - 3.2 Low-Level
 - 3.2.1 Error Codes

- 3.2.2 Result Types
 - 3.2.2.1 Input and Output Ranges
 - 3.2.2.2 Implementation Challenge: Ranges are not the Sum of their Parts
 - 3.2.2.3 Error Handling: Allow All The Options
- 3.2.3 The Encoding Object
 - 3.2.3.1 Encodings Provided by the Standard
 - 3.2.3.2 UTF Encodings: variants?
 - 3.2.3.3 Encoding Schemes: Byte-Based
- 3.2.4 Stateful Objects, or Stateful Parameters?
 - 3.2.4.1 Self-Synchronizing State
 - 3.2.4.2 Empty State and learning from Minimal Allocators
- 3.3 The Need for Speed
 - 3.3.1 Transcoding Compatibility
 - 3.3.2 Eager, Fast Functions with Customizability

4 Implementation

5 Acknowledgements

References

Informative References

§ 1. Revision History

§ 1.1. Revision 0 - June 17th, 2019

— Initial release of exploratory paper.

§ 2. Motivation

It's 2019 and Unicode is still barely supported in both the C and C++ standards.

From the POSIX standard requiring a single-byte encoding by default, heavy limitations placed in `codecv` facets in C and C++, and the utter lack of UTF8/16/32 multi-unit conversion functions by the standard, the programming languages that have shaped the face of development in operating systems, embedded devices and mobile applications has pushed forward a world that is incredibly

unfriendly to a world of text beyond ASCII English. Developers frequently roll their own solutions, and almost every major codebase -- from Chrome to Firefox, Qt to Copperspice, and more -- all have their own variations of hand-crafted text processing. With no standard implementation in C++ and libraries split between various third party implementations plus ICU, it is increasingly difficult and error-prone to handle what is the basic means of communication between people on the planet using C++.

This paper aims to explore the design space for both extremely high performing transcoding (encoding and decoding) as well as a flexible one-by-one interface for more careful and meticulous text processing. This proposal arises from industry experience in large codebases and best-practice open source explorations with [\[libogonek\]](#), [\[icu\]](#), [\[boost.text\]](#) and [\[text_view\]](#) while also building on the concepts and design choices found in both [\[range-v3\]](#) and provably fast text processing such as Windows's WideCharToMultiByte interfaces, *nix utility iconv, and more.

The ultimate goal is to allow an interface that is correct by default but capable of being fast both by Standard Library implementer efforts but also program specialization-friendly free functions. It will produce both an interface for encoding and decoding.

§ 3. Design

The current design has been the culmination of a few years of collaborative and independent research, starting with the earliest papers from Mark Boyall's [\[n3574\]](#), Tom Honermann's [\[p0244r2\]](#), study of ICU's interface, and finally the musings, experience and work of R. Martinho Fernandes in [\[libogonek\]](#). Current and future optimizations are considered to ensure that fast paths are not blocked in the interface proposed for standardization. With [\[boost.text\]](#) with hammering down the internally used encoding to be UTF8, Markus Sherer's participation in SG16 meetings and Bob Steagall's [work in writing a fast UTF8 decoder](#) this paper absorbs a wealth of knowledge to get reach a flexible interface that enables high-throughput.

In reading, implementing, working with and consuming all of these designs, the author of this paper, independent implementers, and several SG16 members have come to the following core tenants:

- strong types for code units allow selecting proper default encodings for these interfaces;
- iterators and ranges are a huge interface win for working with text but are impossible to provide the fastest possible way to encode/decode/transcode text;
- and, avoid creating new vocabulary: improve working with original containers and imposing well-formedness constraints upon them rather than designing new containers from the ground up.

Given these tenants, the following interface choices have arisen for this paper. Each section will describe a piece of the interface, its goals, and how it works. We start first with the high-level low-level encoding interface and its plumbing and core types.

§ 3.1. High Level

Working with the lower level facilities for text processing is not a pretty sight. [Consider the usage of the low-level facilities laid out below](#):

```
std::text::utf8 encoding;

std::text::unicode_code_point array_output[41]{};
std::text::utf8::state encoding_state{};
std::u8string_view input = u8"ΜΑΓ ΓΛΕΣ ΪΤΑΝ, ΝΙ ΜΙΣ ΥΠ ΝΘΑΝ ΒΡΙΓΓΙΦ.";
std::u8string_view working_input = input;
std::span working_output = std::span(array_output);
for (;;) {
    auto result = encoding.decode(working_input, working_output,
                                   encoding_state, std::text::default_error_handler{});
    if (result.error_code != encoding_errc::ok) {
        break;
    }
    if (std::empty(result.input)) {
        break;
    }
    working_input = std::move(result.input);
    working_output = std::move(result.output);
}
assert(std::u32string_view(array_output) == U"ΜΑΓ ΓΛΕΣ ΪΤΑΝ, ΝΙ ΜΙΣ ΥΠ ΝΘΑΝ ΒΡΙΓΓΙΦ.");
```

These low-level facilities -- while powerful and customizable -- do not represent what the average user will -- or should -- be wrangling with. Therefore, the higher-level facilities become incredibly pressing to make these interfaces palatable and sustainable for developers in both the short and long term. Consider the same encoding functionality, boiled down to something far easier to use:

```
std::u32_string output = std::text::decode(u8"ΜΑΓ ΓΛΕΣ ΪΤΑΝ, ΝΙ ΜΙΣ ΥΠ ΝΘΑΝ ΒΡΙΓΓΙΦ.");
assert(output == U"ΜΑΓ ΓΛΕΣ ΪΤΑΝ, ΝΙ ΜΙΣ ΥΠ ΝΘΑΝ ΒΡΙΓΓΙΦ.");
```

This is much simpler and does exactly the same as the above, without all the setup and boilerplate. Of course, taking only the input and giving the output is too **much** of a simplification, so there are a few overloads and variants that will be offered. Namely:

```
namespace std { namespace text {  
  
    template <typename Input, typename Output, typename Encoding,  
              typename State, typename ErrorHandler>  
    constexpr void decode_into(Input&& input, Output&& output,  
                               Encoding&& encoding, State& state, ErrorHandler&& error_har  
  
    template <typename Input, typename Encoding,  
              typename State, typename ErrorHandler>  
    auto decode(Input&& input, Encoding&& encoding,  
                State& state, ErrorHandler&& error_handler);  
  
    template <typename Input, typename Encoding,  
              typename ErrorHandler>  
    auto decode(Input&& input, Encoding&& encoding,  
                ErrorHandler&& error_handler);  
  
    template <typename Input, typename Encoding>  
    auto decode(Input&& input, Encoding&& encoding);  
  
    template <typename Input>  
    auto decode(Input&& input);  
  
}}
```

Similarly named functions for encoding (`std::text::encode`) and transcoding (`std::text::transcode`) will be provided. The point of these functions is eager transformation of the source to the destination. It will also convert all available code points, meaning that it will only stop if the [error_handler parameter forces it to stop](#). On top of eagerly consuming free functions, there needs to be views that allow a person to walk some view of storage with a specified encoding. These encoding views will be called `std::text_view`. Their goal is to provide encoding-agnostic iterators and comparison, as well as some degree of text normalization as a base-line:

```

std::u8text_view my_utf8_text(
    u8"தேமதுரத் தமிழோசை உலகமெலாம் பரவும்வகை செய்தல் வேண்டும்."
);
std::u16text_view my_utf16_text(
    u"தேமதுரத் தமிழோசை உலகமெலாம் பரவும்வகை செய்தல் வேண்டும்."
);
std::u32text_view my_utf32_text(
    U"தேமதுரத் தமிழோசை உலகமெலாம் பரவும்வகை செய்தல் வேண்டும்."
);
assert(my_utf8_text == my_utf16_text);
assert(my_utf16_text == my_utf32_text);
assert(my_utf32_text == my_utf8_text);

```

But how do we build these higher-level functions and views? The answer to that question is going to be the primary exploration of this paper: the low-level details of creating the above higher-level functions and views for encoding. Following sufficient progress of encodings, this paper will then address the needs of normalization.

§ 3.2. Low-Level

The high-level interfaces must be built on *something*: it cannot be magically willed into existence. There is quite a bit of plumbing that goes into the low-level interfaces, most of which will be boilerplate to users but will serve keen use and importance to several library developers and standard library implementers.

§ 3.2.1. Error Codes

There is some boilerplate that needs to be taken care of before we begin building our encoding, decoding, transcoding and similar functionality. First and foremost is the error codes and result types that will go in and out of our encoding functions. The error code enumeration is

`std::text::encoding_errc`. It lists all the reasons an encoding or decoding operation can fail:

```

namespace std { namespace text {

    enum class encoding_errc : int {
        // just fine
        ok = 0x00,
        // input contains ill-formed sequences
        invalid_sequence = 0x01,
        // input contains incomplete sequences
        incomplete_sequence = 0x02,
        // input contains overlong encoding sequence
        // (e.g. for utf8)
        overlong_sequence = 0x03,
        // output cannot receive all the completed
        // code units
        insufficient_output_space = 0x04,
        // sequence can be encoded but resulting
        // code point is invalid (e.g., encodes a lone surrogate)
        invalid_output = 0x05,
        // leading code unit is wrong
        invalid_leading_sequence = 0x06,
        // leading code units were correct, trailing
        // code units were wrong
        invalid_trailing_sequence = 0x07
    };

}}

```

The comments give some small amount of examples about what each one means. The reason 0 is used to signal success is very simple: the next part of the API creates an `encoding_error_category` class and hooks up the machinery for a `std::error_condition`:

```

namespace std {

    template <>
    class is_error_condition_enum< encoding_errc > : true_type {};

    class encoding_error_category : public error_category {
    public:
        constexpr encoding_error_category() noexcept;

        virtual const char* name() const noexcept override;
        virtual string message(int condition) const override;
    };

}

```

This allows the creation of a `std::error_condition`, which is used to signal platform-independent error codes.

§ 3.2.2. Result Types

The result types are the glue that help users who use the low level interface loop through their text properly. It returns updated ranges of both the input and output to indicate how far things have been moved along, on top of an `error_code` and whether or not the result came from an error being handled:


```

namespace std { namespace text {

    template <typename Input, typename Output, typename State>
    class encode_result {
        Input input;
        Output output;
        State& state;
        encoding_errc error_code;
        bool handled_error;

        template <typename InRange, typename OutRange, typename EncodingState>
        constexpr encode_result(InRange&& input, OutRange&& output,
                                EncodingState&& state, encoding_errc error_code = encoding_errc())
            : input(input), output(output), state(state), error_code(error_code), handled_error(false) {}

        template <typename InRange, typename OutRange, typename EncodingState>
        constexpr encode_result(InRange&& input, OutRange&& output,
                                EncodingState&& state, encoding_errc error_code, bool handled_error)
            : input(input), output(output), state(state), error_code(error_code), handled_error(handled_error) {}

        constexpr std::error_condition error() const;
    };

    template <typename Input, typename Output, typename State>
    class decode_result {
        Input input;
        Output output;
        State& state;
        encoding_errc error_code;
        bool handled_error;

        template <typename InRange, typename OutRange, typename EncodingState>
        constexpr decode_result(InRange&& input, OutRange&& output,
                                EncodingState&& state, encoding_errc error_code = encoding_errc())
            : input(input), output(output), state(state), error_code(error_code), handled_error(false) {}

        template <typename InRange, typename OutRange, typename EncodingState>
        constexpr decode_result(InRange&& input, OutRange&& output,
                                EncodingState&& state, encoding_errc error_code, bool handled_error)
            : input(input), output(output), state(state), error_code(error_code), handled_error(handled_error) {}

        constexpr std::error_condition error() const;
    };

}}

```

There is a lot to unpack here. There are two essentially identical structures: `std::encode_result` and `std::decode_result`. These contain the input range, the output range, a reference to the encoding's current state, the error code and whether or not the error handler was invoked. The `bool error_handled` is important because some error handlers may change the `error_code` member to `std::text::encoding_errc::ok`, indicating that things are fine (e.g., a replacement character was successfully inserted into the output stream to replace a bad character).

Having 2 differently-named types with much the same interface is paramount to allow an `error_handler` callable to know how to interpret some errors and whether to try to insert code units into the output stream or code points into the output stream (encoding means code units into output, decoding means code points into the output). If the structures were merged, we would lose this information at compile-time and have to attempt to coerce that information out by examining the `value_type` and `reference` types of the output range. Unfortunately, it is not foolproof because neither the input range or output ranges need to exactly dereference to exactly `Encoding::code_unit` or `Encoding::code_point` types, just things convertible to / from them.

To start, let's examine the `input` and `output` ranges.

§ 3.2.2.1. *Input and Output Ranges*

These are essentially the ranges moved forward as much or as little as the encoding needed to for reading from the input, converting, and writing to the output. It also solves the problem of obtaining maximal speed based on checking if the destination is filled or if the input is exhausted: `unbounded_view` works well since its comparison sentinel always returns the literal "false" bool on comparison, meaning that any compiler beyond the typical `-O0` / `/O0` / etc. levels of optimization will cull those branches of code out.

The decoding result and encoding result types both return the input and output range specified in the structure itself. This represents the changed ranges. Unfortunately, problems arise when one assumes that a range can be reconstructed from its `begin(rng)` and `end(rng)` iterator.

§ 3.2.2.2. *Implementation Challenge: Ranges are not the Sum of their Parts*

Ranges do not offer a generic way to reconstruct themselves from their bits. If you deconstruct a range with `std::ranges::begin` and `std::ranges::end`, the two resulting iterators cannot be put back together again for all ranges. Even ranges which can conceptually handle this are [missing constructors which allow for this](#), and since it is not part of the general interface there is no generic way to do this. However, it would be somewhat silly to lose some of the interface members

and properties of the original class if it does indeed contain a way to handle it: thusly, later iterations of this proposal will likely introduce a `ReconstructibleRange` concept, which will specify that `Range(Iterator<Range>, Sentinel<Range>)` is a valid expression. In cases where it is not, this paper will either check for `Range(Iterator<Range>)` being a valid expression and otherwise fallback to `sub_view<Iterator<Range>, Sentinel<Range>>(Iterator<Range>, Sentinel<Range>)` as the type that goes into the `decode_result`.

§ 3.2.2.3. Error Handling: Allow All The Options

This is a low-level interface. As such, we need a way to accommodate different error handling strategies. There are several ways to report errors used in both the C and C++ standard libraries, from throwing errors, to `error_code` out parameters, to integral return values and even complex return structures. Choosing a scheme here is difficult given the large breadth and depth of error handling history in C++, and while the standard library shows a clear bias towards throwing exceptions it would not be prudent to throw all the time: it may exclude hard and soft real-time programming environments wherein these encoding structures will be needed,.

Error reporting will be done through an error handler to accommodate multiple, which can be any type of callable that matches the desired interface. The standard will provide 3 of these error handlers:

```
namespace std { namespace text {  
  
    class replacement_character_handler;  
    class throw_handler;  
    class assume_valid_handler;  
  
    using default_error_handler = replacement_character_handler;  
  
}}
```

The interface for an error handler will look as such:

```

namespace std { namespace text {

    class an_error_handler {
        template <typename Encoding, typename InputRange,
            typename OutputRange, typename State>
        constexpr auto operator()(const Encoding& encoding,
            encode_result<InputRange, OutputRange, State> result) const
            /* morph result or throw error */
            return result;

    }

    template <typename Encoding, typename InputRange,
        typename OutputRange, typename State>
    constexpr auto operator()(const Encoding& encoding,
        decode_result<InputRange, OutputRange, State> result) const
        /* morph result or throw error */
        return result;

    }

};

}}

```

The implementation is a value-based one, wherein the `current_result` is taken from the implementation of `encode` or `decode` function on the encoding object that puts together its current progress in the form of the current state of the forward-moved input range, the current state of the forward-moved output range, a reference to the current state, and the type of error encountered according to the `std::encoding_errc`. The error handler is then responsible for performing any modifications it wants to the result type, before returning the modified result to be propagated back by the encoding interface.

There are a few things that can be done in the `/* morph result or throw error */` part of that example error handler definition. First and foremost is that someone could look at `current_result.error()` and simply throw a hand-tailored exception. This would bubble out of the function and let the caller decide what to do. Throwing is **explicitly not recommended by default** by prominent vendors and implementers (Mozilla, Apple, the Unicode Consortium, WHATWG, etc.). The recommendation is a good one, because ill-formed text is common and is also the most frequent kind of user input. It is extremely easy to provoke a Denial of Service Attack (DoS Attack) if an application throws an error on malformed input that the application author did not consider.

The default selection of error handler will be the

`std::text::replacement_character_handler`. The `replacement_character_handler` will look inside `Encoding` to see if the expression `Encoding::replacement_code_point` or `Encoding::replacement_code_unit` is well-formed. If so, that character will attempt to be inserted into the `output` range and the error code on the result will be corrected to say "everything is fine" (`std::text::encoding_errc::ok`) and then returned from the function. This follows the Unicode Consortium's and many, many vendors explicit recommendations.

For performance reasons and flexibility, the error callable must have a way to ensure that the user and implementation can agree on whether or not we invoke Undefined Behavior and assume that the text is valid. [\[libogonek\]](#) made an object of type `assume_valid_t`. This paper provides the same here: an error handler of `assume_valid_handler` means that the implementation will eliminate all of its checks and subsequent calls to the error handling interface. A trait will be provided to check if an error handler is ignorable: `std::is_ignorable_error_handler_v<Handler>`. A user can opt into this, but it will not be the default and will require explicit passing of such an error handler to use.

§ 3.2.3. The Encoding Object

Given our result types and error handlers, we can now define the interface for the encoding object itself. Here is the example encoding:

```

namespace std { namespace text {

    // NOTE: exemplary encoding
    // for expository purposes
    // containing all the types
    class example_locale_encoding {
        class __ex_state {
            std::mbstate_t multibyte_state;
        };
        using code_point = char32_t;
        using code_unit = char;
        using state = __ex_state;
        static constexpr size_t max_code_unit_sequence = MB_LEN_MAX;
        static constexpr size_t max_code_point_sequence = 1;

        // optional
        using is_encoding_injective = std::false_type;
        // optional
        using is_decoding_injective = std::true_type;
        // optional
        code_point replacement_code_point = '0xFFFD';
        // optional
        code_unit replacement_code_unit = '?';

        // encodes exactly one full code unit sequence
        // into one full code point sequence
        template <typename In, typename Out, typename Handler>
        encode_result<In, Out, state> encode(
            In&& in_range,
            Out&& out_range,
            state& current_state,
            Handler&& handler
        );

        // decodes exactly one full code point sequence
        // into one full code unit sequence
        template <typename In, typename Out, typename Handler>
        decode_result<In, Out, state> decode(
            In&& in_range,
            Out&& out_range,
            state& current_state,
            Handler&& handler
        );
    };
}
}

```

```
static void reset(state&);  
};  
}}
```

There are many pieces of this encoding object. Some of them fit the purposes explained above. As an overview:

- `code_unit` and `code_point` type definitions let us know what an Encoding's inputs and outputs will be from its functions. It also helps us tell if 2 encodings can be transcoded from one another by having at least the `code_point` in common.
- `is_encoding_injective` and `is_decoding_injective` tells us whether or not the encode or decode operations provide a lossless map from the `code_point` to `code_unit` or vice-versa, respectively.
- `state` allows a user to instantiate the type and control its parameters. At the very least, `Encoding::state` must be default-constructible and its associated encoding must have a `reset` function which will put the `state` in its original, unprocessed internal representation.
- `max_code_unit_sequence` and `max_code_point_sequence` represent integral values which inform users of the encoding the necessary size of a buffer to handle at least one full, encoded sequence of code units and one full, decoded sequence of code points. In most cases, `max_code_point_sequence` will be 1. If the `code_point` type names a Unicode type, it will be 1.

§ 3.2.3.1. Encodings Provided by the Standard

The primary reason for the standard to provide an encoding is to ensure that it produces a way for applications to communicate with one another. As a baseline, the standard should support all the encodings it ships with its string literal types. On top of that, there is an important base-level optimization when working with strictly ASCII text that can be implemented with UTF8 which would most library implementers are interested in shipping. This means that the following encodings will be shipped by the standard library:

```

namespace std { namespace text {

    class ascii;
    class utf8;
    class utf16;
    class utf32;
    class narrow_execution;
    class wide_execution;

}}

```

The first four structures correspond directly to what they name. The last two structures are specific, key encodings for interoperating with locale-dependent narrow execution encoding data as well as locale-dependent wide execution encoding data. It is imperative the standard ships these because only the implementation knows the runtime execution encoding. The case is similar for the wide execution encoding. These 6 are the total of all that at the bare minimum must be shipped with the standard. `ascii` holds a special place here because it is a direct subset of `utf8`. If an individual knows their text is in purely ASCII ahead of time and they work in UTF8, this information can be used to bit-blast (`memcpy`) the data from UTF8 to ASCII.

§ 3.2.3.2. UTF Encodings: variants?

There are many variants of encodings like UTF8 and UTF16. These include [\[wtf8\]](#) or [\[cesu8\]](#) and are useful for internal processing and interoperability with certain systems, like direct interfacing with Java or communication with an Oracle database. However, almost none of these are publicly recommend as interchange formats: both CESU-8 and WTF-8 are documented and used internally for legacy reasons. In some cases, they also represent security vulnerabilities if they are used in interchange for the internet. This makes them less and less desirable to provide VIA the standard. However, it is worth acknowledging that supporting WTF-8 and CESU-8 as encodings will ease individuals who need to roll such encodings for their applications.

More pressingly, there is a wide body of code that operates with `char` as the code unit for their UTF8 encodings. This is also subtly wrong, because on a handful of systems `char` is not unsigned, but signed. Math and bit characteristics for these types are wrong for the typical operations performed in UTF8 encoders and decoders (and many people -- including Markus Schröder that spends a lot of time with ICU -- just wish `char` was unsigned since it would have saved a lot of time from bugs). On one hand, providing variants that allow someone to pick something like the code unit for UTF16 or UTF8

would make it easier to have text types which play nice with the Windows APIs or existing code bases. The interface would look something like this...

```
namespace std { namespace text {  
  
    template <typename CharT, bool encode_null, bool encode_lone_surrogates>  
    class basic_utf8;  
  
    using utf8 = basic_utf8<char8_t, false, false>;  
  
    template <typename CharT, bool allow_lone_surrogates>  
    class basic_utf16;  
  
    using utf16 = basic_utf8<char16_t, false>;  
  
}}
```

And externally, libraries and applications could add their own using statements and type definitions for the purposes of internal interoperation:

```
namespace my_app {  
  
    using compat_utf8 = std::basic_utf8<char>;  
    using filesystem16 = std::basic_utf16<wchar_t, true>;  
  
}
```

There is clear utility that can be had here. But, this is not going to be looked into too deeply for the first iteration of this proposal.

§ 3.2.3.3. Encoding Schemes: Byte-Based

Unicode specifies what are called Encoding Schemes for the encodings whose code unit size exceeds a single byte. This is essentially UTF16 and UTF32, of which there is UTF16 Little Endian (UTF16-LE), UTF16 Big Endian (UTF16-BE), UTF32 Little Endian (UTF32-LE), and UTF32 Big Endian (UTF32-BE). Encoding schemes can be generically handled without creating extremely specific encodings by creating an `encoding_scheme<...>` template. It will look much like so:

```

namespace std { namespace text {

    template <std::endian endianness, typename Encoding, typename Byte
    class encoding_scheme;

}}

```

This is a transformative encoding type that takes the source (network) endianness and translates it to the native (host) endianness. It has an identical interface to the `Encoding` type passed in, with the caveat that the `code_unit` member type is the same as `Byte`. Really, all it does is call the same `encode` or `decode` function with small wrappers around the passed-in ranges that takes bytes and composes them into the internal `Encoding::code_unit` type, or when writing out takes an `Encoding::code_unit` type and writes it out into its byte-based form. A few SG16 members have frequently advocated that the base input and outputs for all types matching the `Encoding` concept should be byte-based.

This paper disagrees with that supposition and instead goes the route of providing a wrapping encoding scheme. The benefit here is flexibility and independence from byte ordering at the `Encoding` level: the `encoding_scheme` becomes the layer at which such a concern is both concentrated and isolated. Now, *no* encoding needs to duplicate its interface at all, while still retaining strong and separately named types that one can perform additional optimization on. This has also already seen implementation experience in [\[libogonek\]](#)'s [\[libogonek-encoding_scheme\]](#) type, with no qualms from users.

§ 3.2.4. Stateful Objects, or Stateful Parameters?

Stateful objects are good for encapsulation, reuse and transportation. They have been proven in many APIs both C and C++ to provide a good, reentrant API with all relevant details captured on the (sometimes opaque) object itself. After careful evaluation, stateful parameter rather than a wholly stateful object for the function calls in encoding and decoding types are a better choice for this low-level interface. The main and important benefits for having the state be passed to the encoding / decoding function calls as a parameter are that it:

- maintains that encoding objects can be cheap to construct, copy and move;
- improves the general reusability of encoding objects by allowing state to be massaged into certain configurations by users;

— and, allows users to set the state in a public way without having to prescribe a specific API for all encoders to do that.

The reason for keeping encoding types cheap is that they will be constructed, copied, and moved a lot, especially in the face of the ranges that SG16 is going to be putting a lot of work into (`std::text_view<View, Encoding, ...>`). Ranges require that they can be constructed in (amortized) constant time; this change allows us to shift the construction for what may be potentially expensive state to other places.

As a poignant example: consider the case of execution encoding character sets today, which often defer to the current locale. Locale is inherently expensive to construct and use: if the standard has to have an encoding that grabs or creates a `codecvt` or `locale` member, we will immediately lose a large portion of users over the performance drag during construction of higher-level abstractions that rely on the encoding. It is also notable that this is the same [mistake std::wstring_convert shipped with](#) and is one of the largest contributing reasons to its lack of use and subsequent deprecation (on top of its poor implementation in several libraries, from the VC++ standard library to libc++).

In contrast, consider having an explicit parameter. At the cost of making a low-level interface take one more parameter, the state can be paid for once and reused in many separate places, allowing a user to front-load the state's expenses up-front. It also allows the users to set or get the locale ahead of time and reuse it consistently. It also allows for encoding or decoding operations to be reused or restart in the cases of interruptible or incomplete streams, such as network reading or I/O buffering. These are potent use cases wherein such a design decision becomes very helpful.

§ 3.2.4.1. Self-Synchronizing State

A self-synchronizing code is a uniquely decodable source symbol stream whose output provides a direct and unambiguous mapping with the source symbol stream. These require no state to parse given a sequence, because a sequence must be either valid or invalid with no intermediate states of "potentially valid". For example, not fully decoding any of the Unicode Transformation Formats's code units into a single code point -- unfinished surrogates or half-delivered byte sequences -- in full is an error because no sub-sequence can identify another code point. This is the primary usage of stateful encoding and decoding operations: tracking what was last seen -- among other parameters -- for the purposes of disambiguating incoming input.

If an encoding is self-synchronizing, then at no point is there a need to refer to an "potentially correct but need to see more" state: the input is either wholly correct, or it is not. Therefore, an encoding is considered self-synchronizing *by default* if it's state parameter is empty (i.e. `std::is_empty_v<state>` is true). Note that the inverse cannot be assumed to be true: if a state

object is not empty, it can still be self-synchronizing. The implementation just cannot assume so, and thusly must treat the state parameter by-default as non-self-synchronizing.

Thusly, the trait `std::is_self_synchronizing<T>` will give users a way to avoid needing to have to inspect the `state` at all. This trait eliminates the need to worry about shift states or other hidden shenanigans in the encoding and decoding operations, simplifying error handling. In the case of a stateful but self-synchronizing state, one must override the trait

`std::is_self_synchronizing_v<T>` to declare their state by-default self-synchronizing.

§ 3.2.4.2. Empty State and learning from Minimal Allocators

If `std::is_empty_v<State>` is true, then there is no reason to require that the state is passed to the encoding functions. This is more or less an API "kindness", but so long as the state is an empty object it does not have to be passed to the `encode` or `decode` functions. This is not going to be proposed at this time, but for API usability it should be looked into later in the life of this proposal (e.g., revision 2).

§ 3.3. The Need for Speed

Correctness is correctness. So is performance. If these methods and the resulting interface are not fast enough to meet the needs of the programmers, there will be little to no adoption. Thanks to work by Bob Steagall and Zach Laine, we know for a fact that it is incredibly hard -- perhaps even impossible - - to make a range-based or iterator-based interface which will achieve the text processing speeds that will satisfy users. There shall be no room for a lower level abstraction or language here, and the first steps to doing that are recognizing the benefits of eager encoding, decoding and transcoding interfaces.

§ 3.3.1. Transcoding Compatibility

A set of program-overrideable traits will be provided to clue implementations in on the ability to trivially relocate/trivially copy data from source to destination with respect to encodings. This is done primarily because of cases where one encoding is a strict superset or subset of another encoding. For example, ASCII encodings are a subset of UTF8 encodings, and in general allow someone to strictly `memcpy` the bits from one storage to the other without loss of information. Therefore, there will be a trait that specifies transcoding compatibility named `std::is_bitwise_compatible_encoding_v<From, To>`. This will allow implementations to use `std::copy` directly when going from one encoding to another, rather than round-tripping through the common `code_point` type and some small intermediate storage.

§ 3.3.2. Eager, Fast Functions with Customizability

Research and implementation experience with [\[boost.text\]](#), [\[text_view\]](#) and others has made it plainly clear that while iterators and ranges can produce an extremely efficient binary, it is still not the fastest code that can be written to compete with hand-written or vectorized text processing routines made specifically for each encoding. Therefore, it is imperative that lazy ranges cannot be the only solution if we want the standard to steadily and nicely supplant the codebase-specific or ad-hoc solutions individuals keep rolling for encoding and decoding operations.

Considering this is going to be one of the most fundamental text layers that sits between typical text and a lot of the new I/O routines, it is imperative that these conversions and transcodes are not only as fast as possible, but customizable. The user can already customize the encoding by creating their own conforming encoding object, but encodings still do their transformations on a code point-by-code point basis. Therefore, a means of extensibility needs to be chosen for the `std::text::encode`, `std::text::decode` and `std::text::transcode` functions. As this paper is targeting C++23, we hope that Matt Calabrese's [\[p1292\]](#) receives favor in the Evolution Design Groups so that our extension mechanisms are nice. Failing that, a design similar to `std::ranges`'s customization points -- as laid out in [\[n4381\]](#) -- might be useful here, albeit this is worrisome considering the amount of templated arguments which we do not want to apply overly-restrictive concepts or restraints to. We can also provide a struct that users can use partial template specialization matching and concepts. These are all non-ideal ways of specializing this interface, so we will wait to pick the method of extension.

What is not negotiable is that it must be extensible. Users should be able to write fast transcoding functions that the standard picks up for their own encoding types. From GB1032 to other ISO and WHATWG encodings, there will always be a need to extend the fast bulk processing of the standard.

§ 4. Implementation

While the ideas presented in this paper have been explored in various different forms, the ideas have never been succinctly composed into a single distributable library. Therefore, the author of this paper is working on an implementation that synthesizes all of the learning from [\[icu\]](#), [\[boost.text\]](#), [\[text_view\]](#) and [\[libogonek\]](#).

This paper's r1 hopes to contain benchmarks, initial implementation and usage experience. This paper's r2 hopes to contain more benchmarks, refined implementation and additional field and usage experience after a more valuable and viable minimum product is established. The current implementation is being incubated in [\[phd.text\]](#), but will likely be moved to its own repository soon after the initial implementation for `phd::text_view` and `phd::text` are finished.

§ 5. Acknowledgements

Thanks to R. Martinho Fernandes, whose insightful Unicode quips got me hooked on the problem place many, many years ago and helped me develop my first in-house solution for an encoding container adaptor several years ago. Thanks to Mark Boyall, Xeo, and Eric Tremblay for bouncing off ideas, fixes, and other thoughts many years ago when struggling to compile libogonek on a disastrous Microsoft Visual Studio November 2012 CTP compiler.

Thanks to Tom Honermann, who had me present my second SG16 meeting before it was SG16 and help represent and carry his papers which gave me the drive to help fix the C++ standard for text. Many thanks to Zach Laine, whose tireless implementation efforts have given me much insight and understanding into the complexities of Unicode and whose implementation in Boost.Text made clear the tradeoffs and performance issues. Thanks to Mark Zeren who helped keep me in SG16 and working on these problems.

And thank you to those of you who grew tired of an ASCII-only world and supported this effort.

§ References

§ Informative References

[BOOST.TEXT]

Zach Laine. [Boost.Text](https://github.com/tzlaine/text). October 20th, 2018. URL: <https://github.com/tzlaine/text>

[CESU8]

Unicode Consortium. [UTR #26, Compatibility Encoding Scheme for UTF-16: 8-Bit \(CESU-8\)](https://www.unicode.org/reports/tr26/). March 13th, 2019. URL: <https://www.unicode.org/reports/tr26/>

[FAST-UTF8]

Bob Steagall. [Fast Conversion From UTF-8 with C++, DFAs, and SSE Intrinsics](https://www.youtube.com/watch?v=5FQ87-Ecb-A). September 26th, 2019. URL: <https://www.youtube.com/watch?v=5FQ87-Ecb-A>

[ICU]

Unicode Consortium. [International Components for Unicode](http://site.icu-project.org/). April 17th, 2019. URL: <http://site.icu-project.org/>

[LIBOGONEK]

R. Martinho Fernandes. [Ogonek](https://github.com/libogonek/ogonek). December 9th, 2013. URL: <https://github.com/libogonek/ogonek>

[LIBOGONEK-ENCODING_SCHEME]

R. Martinho Fernandes. [encoding_scheme](https://github.com/libogonek/ogonek/blob/dev/include/ogonek/encoding/encoding_scheme.h%2). December 9th, 2013. URL: https://github.com/libogonek/ogonek/blob/dev/include/ogonek/encoding/encoding_scheme.h%2

[B%2B#L80](#)

[N3574]

Mark Boyall. [Binding stateful functions as function pointers](#). 10 March 2013. URL: <https://wg21.link/n3574>

[N4381]

Eric Niebler. [Suggested Design for Customization Points](#). 11 March 2015. URL: <https://wg21.link/n4381>

[P0244R2]

Tom Honermann. [Text_view: A C++ concepts and range based character encoding and code point enumeration library](#). 13 June 2017. URL: <https://wg21.link/p0244r2>

[P1292]

Matt Calabrese. [Customization Point Functions](#). October 10th, 2018. URL: <https://wg21.link/p1292>

[PHD.TEXT]

ThePhD. [phd::text -- encoding and unicode for C++23](#). June 12th, 2019. URL: <https://github.com/ThePhD/phd/tree/master/include/phd/text>

[RANGE-V3]

Eric Niebler; Casey Carter. [range-v3](#). June 11th, 2019. URL: <https://github.com/ericniebler/range-v3>

[RANGE-V3-SENTINEL-ISSUE]

ThePhD; Eric Niebler. [Ranges which take a sentinel should be constructible from {Iterator, Sentinel}](#). June 11th, 2019. URL: <https://github.com/ericniebler/range-v3/issues/1192>

[SOL2-WSTRING_CONVERT]

ThePhD. [wstring_convert sucks](#). January 27th, 2018. URL: <https://github.com/ThePhD/sol2/issues/571>

[TEXT_VIEW]

Tom Honermann. [text_view](#). November 10th, 2017. URL: https://github.com/tahonermann/text_view

[WTF8]

Simon Sapin. [The WTF-8 encoding](#). September 26th, 2019. URL: <https://simonsapin.github.io/wtf-8/>